

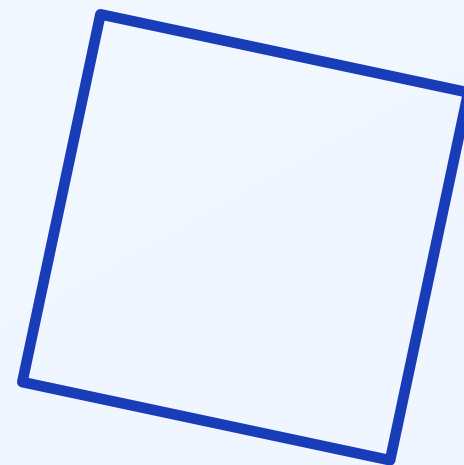
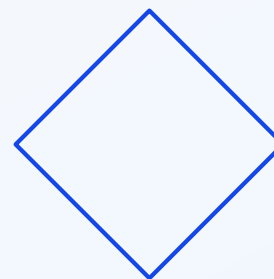
CS464

Data and Network Security

Traditional Block Cipher, DES

Hafiz Syed Ahmed Qasim

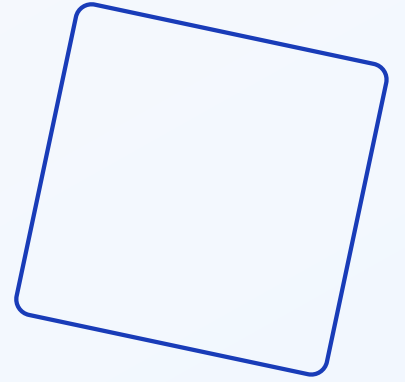
ahmed.qasim@giki.edu.pk



Learning Objectives

After studying this chapter, you should be able to:

- 1 Understand the distinction between stream ciphers and block ciphers.
- 2 Present an overview of the Feistel cipher and explain how decryption is the inverse of encryption.
- 3 Present an overview of Data Encryption Standard (DES).
- 4 Explain the concept of the avalanche effect.
- 5 Discuss the cryptographic strength of DES.
- 6 Summarize the principal block cipher design principles.

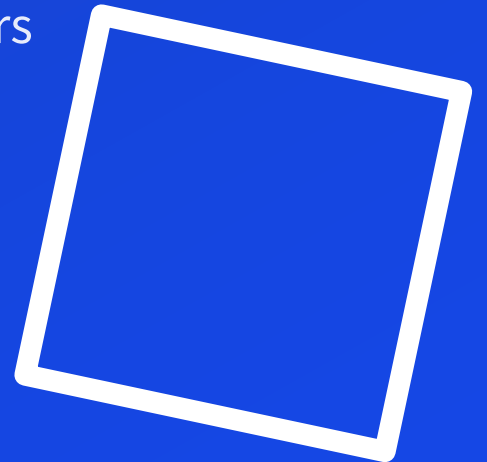




4.1

Traditional Block Cipher Structure

Introduction to the fundamental structure of traditional block ciphers and the Feistel cipher design.



Stream Ciphers and Block Ciphers

Stream Ciphers

A stream cipher is one that encrypts a digital data stream one bit or one byte at a time.

Examples of classical stream ciphers are the autokeyed Vigenère cipher and the Vernam cipher.

In the ideal case, a one-time pad version of the Vernam cipher would be used, in which the keystream (k_i) is as long as the plaintext bit stream (p_i).

If the cryptographic keystream is random, then this cipher is unbreakable by any means other than acquiring the keystream.

However, the keystream must be provided to both users in advance via some independent and secure channel, which introduces insurmountable logistical problems if the intended data traffic is very large.

Practical Stream Cipher Implementation

For practical reasons, the bit-stream generator must be implemented as an algorithmic procedure, so that the cryptographic bit stream can be produced by both users.

In this approach, the bit-stream generator is a key-controlled algorithm and must produce a bit stream that is cryptographically strong.

That is, it must be computationally impractical to predict future portions of the bit stream based on previous portions of the bit stream.

Stream Ciphers and Block Ciphers (Continued)

Block Ciphers

A block cipher is one in which a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length.

Typically, a block size of 64 or 128 bits is used.

As with a stream cipher, the two users share a symmetric encryption key.

Using some of the modes of operation, a block cipher can be used to achieve the same effect as a stream cipher.

Why Focus on Block Ciphers?

Far more effort has gone into analyzing block ciphers.

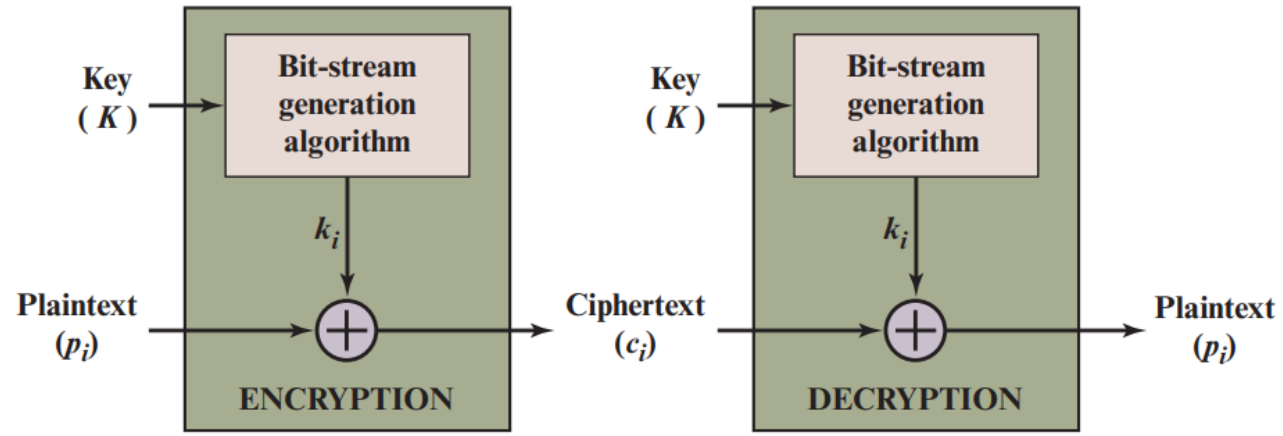
In general, they seem applicable to a broader range of applications than stream ciphers.

The vast majority of network-based symmetric cryptographic applications make use of block ciphers.

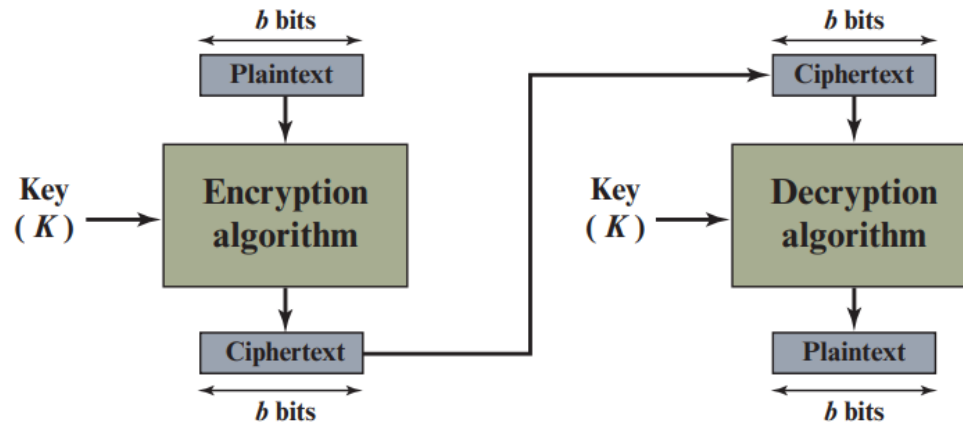
Accordingly, the concern in this chapter, and in our discussions throughout the book of symmetric encryption, will primarily focus on block ciphers.

Figure 4.1

Stream Cipher and Block Cipher



(a) Stream cipher using algorithmic bit-stream generator



(b) Block cipher

Motivation for the Feistel Cipher Structure

Block Cipher Fundamentals

A block cipher operates on a plaintext block of n bits to produce a ciphertext block of n bits.

There are 2^n possible different plaintext blocks and, for the encryption to be reversible (i.e., for decryption to be possible), each must produce a unique ciphertext block.

Such a transformation is called reversible, or nonsingular.

Reversible vs Irreversible Mappings

Example for $n = 2$:

Reversible Mapping:

00 → 11
01 → 10
10 → 00
11 → 01

Irreversible Mapping:

00 → 11
01 → 10
10 → 01
11 → 01

(Ciphertext 01 has two possible plaintexts)

If we limit ourselves to reversible mappings, the number of different transformations is $2^n!$.

The Ideal Block Cipher

General Substitution Cipher

Figure 4.2 illustrates the logic of a general substitution cipher for $n = 4$.

A 4-bit input produces one of 16 possible input states, which is mapped by the substitution cipher into a unique one of 16 possible output states, each of which is represented by 4 ciphertext bits.

The encryption and decryption mappings can be defined by a tabulation, as shown in Table 4.1.

This is the most general form of block cipher and can be used to define any reversible mapping between plaintext and ciphertext.

Feistel's Ideal Block Cipher

Feistel refers to this as the ideal block cipher, because it allows for the maximum number of possible encryption mappings from the plaintext block.

However, there is a practical problem with the ideal block cipher.

If a small block size, such as $n = 4$, is used, then the system is equivalent to a classical substitution cipher.

Such systems are vulnerable to a statistical analysis of the plaintext.

Figure 4.2

General n -bit- n -bit Block Substitution

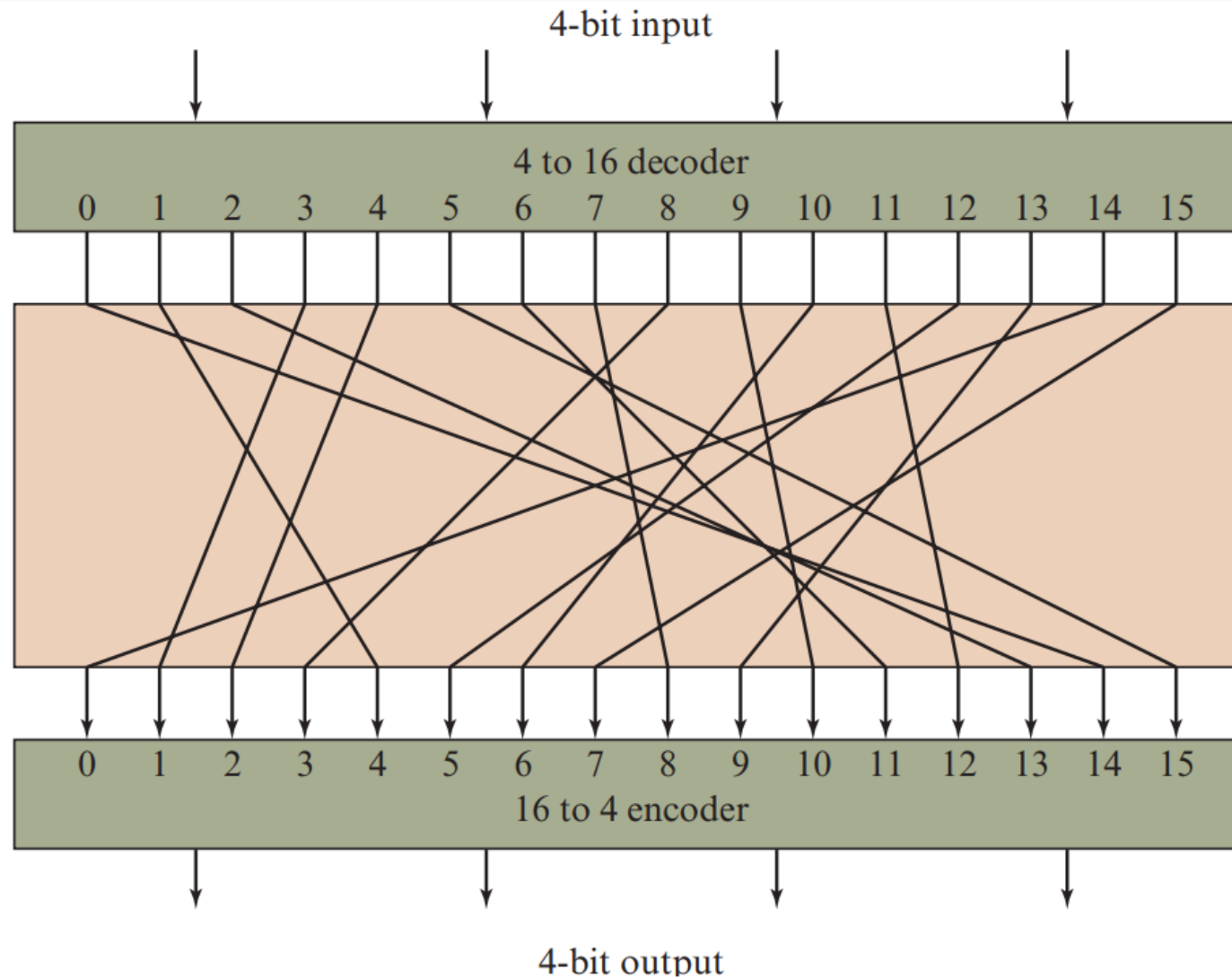


Figure 4.2 General n -bit- n -bit Block Substitution (shown with $n = 4$)



Table 4.1

Encryption and Decryption Tables for Substitution Cipher



Table 4.1 Encryption and Decryption Tables for Substitution Cipher of Figure 4.2

Plaintext	Ciphertext
0000	1110
0001	0100
0010	1101
0011	0001
0100	0010
0101	1111
0110	1011
0111	1000
1000	0011
1001	1010
1010	0110
1011	1100
1100	0101
1101	1001
1110	0000
1111	0111

Ciphertext	Plaintext
0000	1110
0001	0011
0010	0100
0011	1000
0100	0001
0101	1100
0110	1010
0111	1111
1000	0111
1001	1101
1010	1001
1011	0110
1100	1011
1101	0010
1110	0000
1111	0101

Practical Problems with the Ideal Block Cipher

The Key Length Problem

An arbitrary reversible substitution cipher (the ideal block cipher) for a large block size is not practical from an implementation and performance point of view.

For such a transformation, the mapping itself constitutes the key.

Consider Table 4.1, which defines one particular reversible mapping from plaintext to ciphertext for $n = 4$.

The mapping can be defined by the entries in the second column, which show the value of the ciphertext for each plaintext block.

This, in essence, is the key that determines the specific mapping from among all possible mappings.

Key Length Calculations

Using this straightforward method of defining the key, the required key length is $(4 \text{ bits}) \times (16 \text{ rows}) = 64 \text{ bits}$.

In general, for an n -bit ideal block cipher, the length of the key defined in this fashion is $n \times 2^n$ bits.

For a 64-bit block, which is a desirable length to thwart statistical attacks, the required key length is $64 \times 2^{64} = 2^{70} \approx 10^{21}$ bits.

This key length is completely impractical!

Feistel's Solution: Product Cipher

Feistel's Proposal

Feistel points out that what is needed is an approximation to the ideal block cipher system for large n , built up out of components that are easily realizable.

Feistel proposed that we can approximate the ideal block cipher by utilizing the concept of a product cipher.

A product cipher is the execution of two or more simple ciphers in sequence in such a way that the final result or product is cryptographically stronger than any of the component ciphers.

The Goal

The essence of the approach is to develop a block cipher with a key length of k bits and a block length of n bits.

This allows a total of 2^k possible transformations, rather than the $2^n!$ transformations available with the ideal block cipher.

This makes the key size practical while maintaining strong cryptographic properties.

Feistel proposed the use of a cipher that alternates substitutions and permutations.

Substitution and Permutation Operations

Substitution

Each plaintext element or group of elements is uniquely replaced by a corresponding ciphertext element or group of elements.

This operation provides confusion by making the relationship between the key and ciphertext complex.

Permutation

A sequence of plaintext elements is replaced by a permutation of that sequence.

That is, no elements are added or deleted or replaced in the sequence; rather, the order in which the elements appear in the sequence is changed.

This operation provides diffusion by spreading the influence of individual plaintext bits across the ciphertext.

Connection to Shannon's Work

Feistel's approach is a practical application of a proposal by Claude Shannon to develop a product cipher that alternates confusion and diffusion functions.

The Feistel cipher structure dates back over a quarter century and is based on Shannon's proposal from 1945.

This structure is used by many significant symmetric block ciphers currently in use, including TDEA (Triple DES), which is approved by NIST for general use.

Diffusion and Confusion

Shannon's Building Blocks

The terms diffusion and confusion were introduced by Claude Shannon to capture the two basic building blocks for any cryptographic system.

Shannon's concern was to thwart cryptanalysis based on statistical analysis.

Assume the attacker has some knowledge of the statistical characteristics of the plaintext, such as letter frequency distributions or probable words.

Diffusion

In diffusion, the statistical structure of the plaintext is dissipated into long-range statistics of the ciphertext.

This is achieved by having each plaintext digit affect the value of many ciphertext digits.

Generally, this is equivalent to having each ciphertext digit be affected by many plaintext digits.

In a binary block cipher, diffusion can be achieved by repeatedly performing some permutation on the data followed by applying a function to that permutation.

Confusion

The Concept of Confusion

Every block cipher involves a transformation of a block of plaintext into a block of ciphertext, where the transformation depends on the key.

The mechanism of diffusion seeks to make the statistical relationship between the plaintext and ciphertext as complex as possible in order to thwart attempts to deduce the key.

On the other hand, confusion seeks to make the relationship between the statistics of the ciphertext and the value of the encryption key as complex as possible, again to thwart attempts to discover the key.

Achieving Confusion

Even if the attacker can get some handle on the statistics of the ciphertext, the way in which the key was used to produce that ciphertext is so complex as to make it difficult to deduce the key.

This is achieved by the use of a complex substitution algorithm.

In contrast, a simple linear substitution function would add little confusion.

Diffusion and confusion have become the cornerstone of modern block cipher design.

Feistel Cipher Structure

Encryption Structure

The inputs to the encryption algorithm are a plaintext block of length $2w$ bits and a key K .

The plaintext block is divided into two halves, LE_0 and RE_0 .

The two halves of the data pass through n rounds of processing and then combine to produce the ciphertext block.

Each round i has as inputs LE_{i-1} and RE_{i-1} derived from the previous round, as well as a subkey K_i derived from the overall K .

Round Operations

All rounds have the same structure.

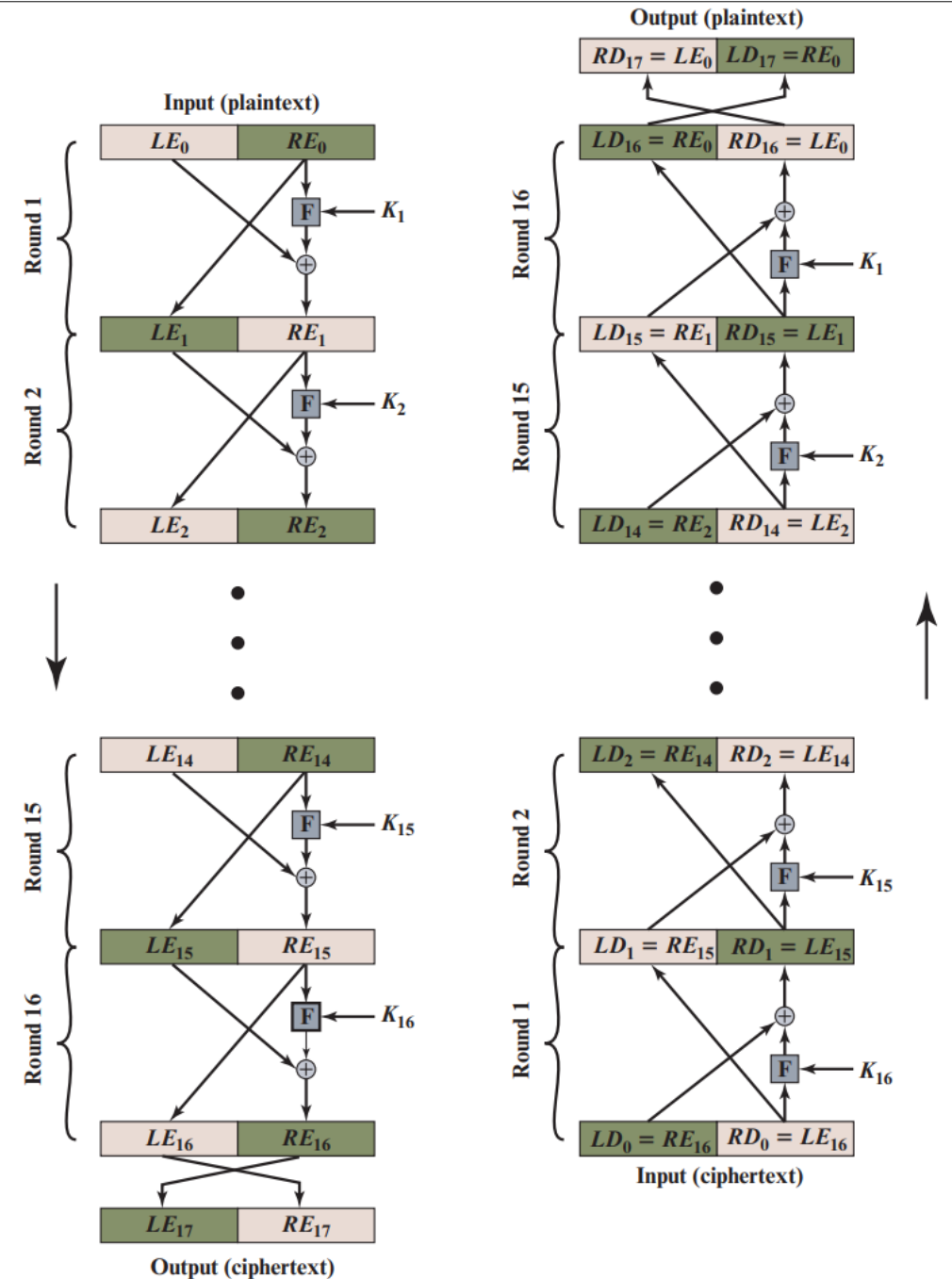
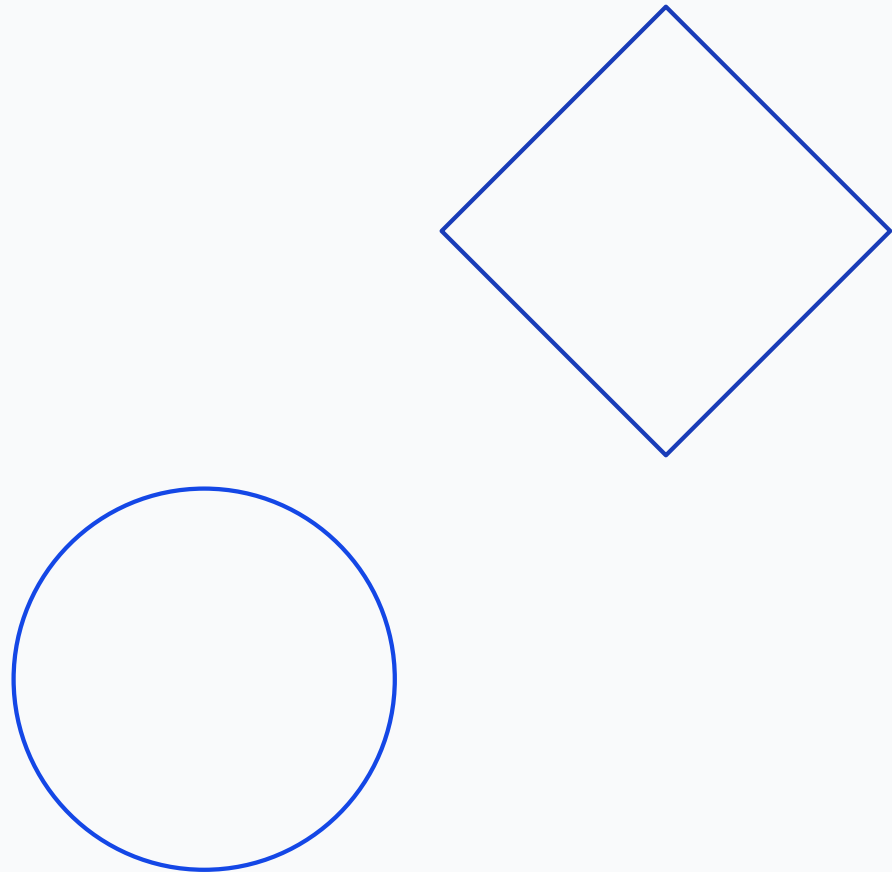
A substitution is performed on the left half of the data by applying a round function F to the right half of the data and then taking the XOR of the output of that function and the left half of the data.

The round function has the same general structure for each round but is parameterized by the round subkey K_i .

Following this substitution, a permutation is performed that consists of the interchange of the two halves of the data.

Figure 4.3

Feistel Encryption and Decryption (16 rounds)



Feistel Cipher Parameters and Design Features

Block Size

Larger block sizes mean greater security (all other things being equal) but reduced encryption/decryption speed.

Greater security is achieved by greater diffusion. Traditionally, 64 bits was common; AES uses 128 bits.

Key Size

Larger key size means greater security but may decrease encryption/decryption speed.

Greater security is achieved by greater resistance to brute-force attacks and greater confusion. 128 bits is now common.

Number of Rounds

The essence of the Feistel cipher is that a single round offers inadequate security but multiple rounds offer increasing security.

A typical size is 16 rounds.

Subkey Generation Algorithm

Greater complexity in this algorithm should lead to greater difficulty of cryptanalysis.

The subkeys K_i are different from K and from each other.

Round Function F

Greater complexity generally means greater resistance to cryptanalysis. The function F provides the element of confusion in a Feistel cipher.

Fast Software Encryption/Decryption: In many cases, encryption is embedded in applications, so speed of execution becomes a concern.

Ease of Analysis: Making the algorithm easy to analyze helps identify cryptanalytic vulnerabilities and develop assurance of its strength.

Feistel Decryption Algorithm

Decryption Process

The process of decryption with a Feistel cipher is essentially the same as the encryption process.

The rule is as follows: Use the ciphertext as input to the algorithm, but use the subkeys K_i in reverse order.

That is, use K_n in the first round, K_{n-1} in the second round, and so on, until K_1 is used in the last round.

This is a nice feature, because it means we need not implement two different algorithms; one for encryption and one for decryption.

Mathematical Verification

To see that the same algorithm with a reversed key order produces the correct result, consider the encryption process:

$$LE_i = RE_{i-1}$$

$$RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i)$$

Rearranging terms:

$$RE_{i-1} = LE_i$$

$$LE_{i-1} = RE_i \oplus F(RE_{i-1}, K_i) = RE_i \oplus F(LE_i, K_i)$$

XOR Properties and Decryption Proof

XOR Properties Used

The XOR operation has the following properties that enable decryption:

$[A \oplus B] \oplus C = A \oplus [B \oplus C]$ (Associative property)

$D \oplus D = 0$ (Self-inverse property)

$E \oplus 0 = E$ (Identity property)

Decryption Verification Example

On the decryption side:

$$LD_1 = RD_0 = LE_{16} = RE_{15}$$

$$RD_1 = LD_0 \oplus F(RD_0, K_{16}) = RE_{16} \oplus F(RE_{15}, K_{16})$$

$$= [LE_{15} \oplus F(RE_{15}, K_{16})] \oplus F(RE_{15}, K_{16})$$

Thus, we have $LD_1 = RE_{15}$ and $RD_1 = LE_{15}$.

Therefore, the output of the first round of the decryption process is $RE_{15} \parallel LE_{15}$, which is the 32-bit swap of the input to the sixteenth round of the encryption.

Figure 4.4

Feistel Example

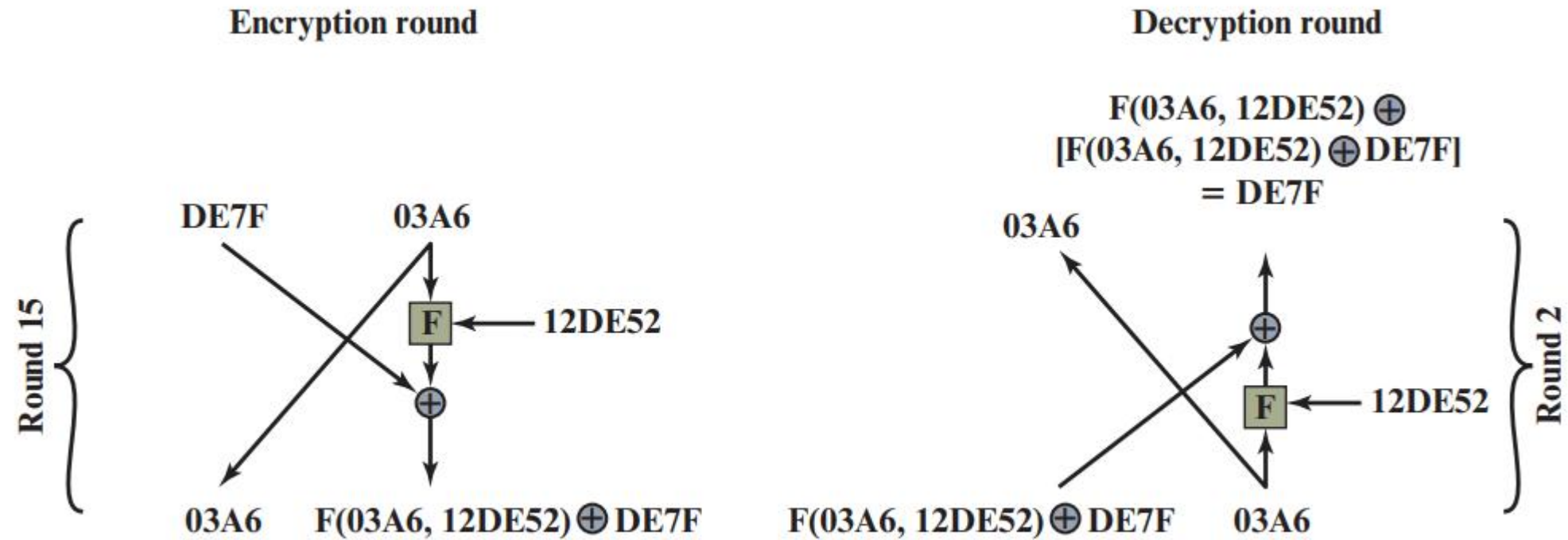
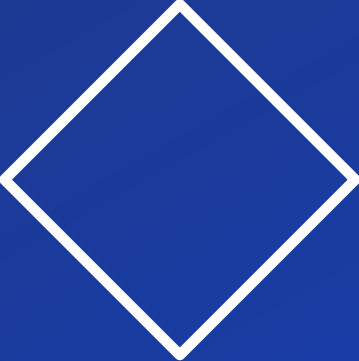


Figure 4.4 Feistel Example



4.2

The Data Encryption Standard



Introduction to DES as the most widely used encryption scheme until AES, issued as FIPS PUB 46 in 1977.

DES Overview and History

Introduction to DES

Until the introduction of the Advanced Encryption Standard (AES) in 2001, the Data Encryption Standard (DES) was the most widely used encryption scheme.

DES was issued in 1977 by the National Bureau of Standards, now the National Institute of Standards and Technology (NIST), as Federal Information Processing Standard 46 (FIPS PUB 46).

The algorithm itself is referred to as the Data Encryption Algorithm (DEA).

DES Technical Specifications

For DEA, data are encrypted in 64-bit blocks using a 56-bit key.

The algorithm transforms 64-bit input in a series of steps into a 64-bit output.

The same steps, with the same key, are used to reverse the encryption.

DES Timeline

Over the years, DES became the dominant symmetric encryption algorithm, especially in financial applications.

In 1994, NIST reaffirmed DES for federal use for another five years; NIST recommended the use of DES for applications other than the protection of classified information.

In 1999, NIST issued a new version of its standard (FIPS PUB 46-3) that indicated that DES should be used only for legacy systems and that triple DES be used.

DES Encryption Process

Three Phases of DES Encryption

- 1 Initial Permutation (IP)**
The 64-bit plaintext passes through an initial permutation that rearranges the bits to produce the permuted input.
- 2 Sixteen Rounds**
A phase consisting of sixteen rounds of the same function, which involves both permutation and substitution functions.
- 3 Final Permutation (IP^{-1})**
The preoutput is passed through a permutation that is the inverse of the initial permutation function, to produce the 64-bit ciphertext.

Key Processing

The right-hand portion shows the way in which the 56-bit key is used.

Initially, the key is passed through a permutation function.

Then, for each of the sixteen rounds, a subkey (K_i) is produced by the combination of a left circular shift and a permutation. The permutation function is the same for each round, but a different subkey is produced because of the repeated shifts of the key bits.

Figure 4.5

General Depiction of DES Encryption Algorithm

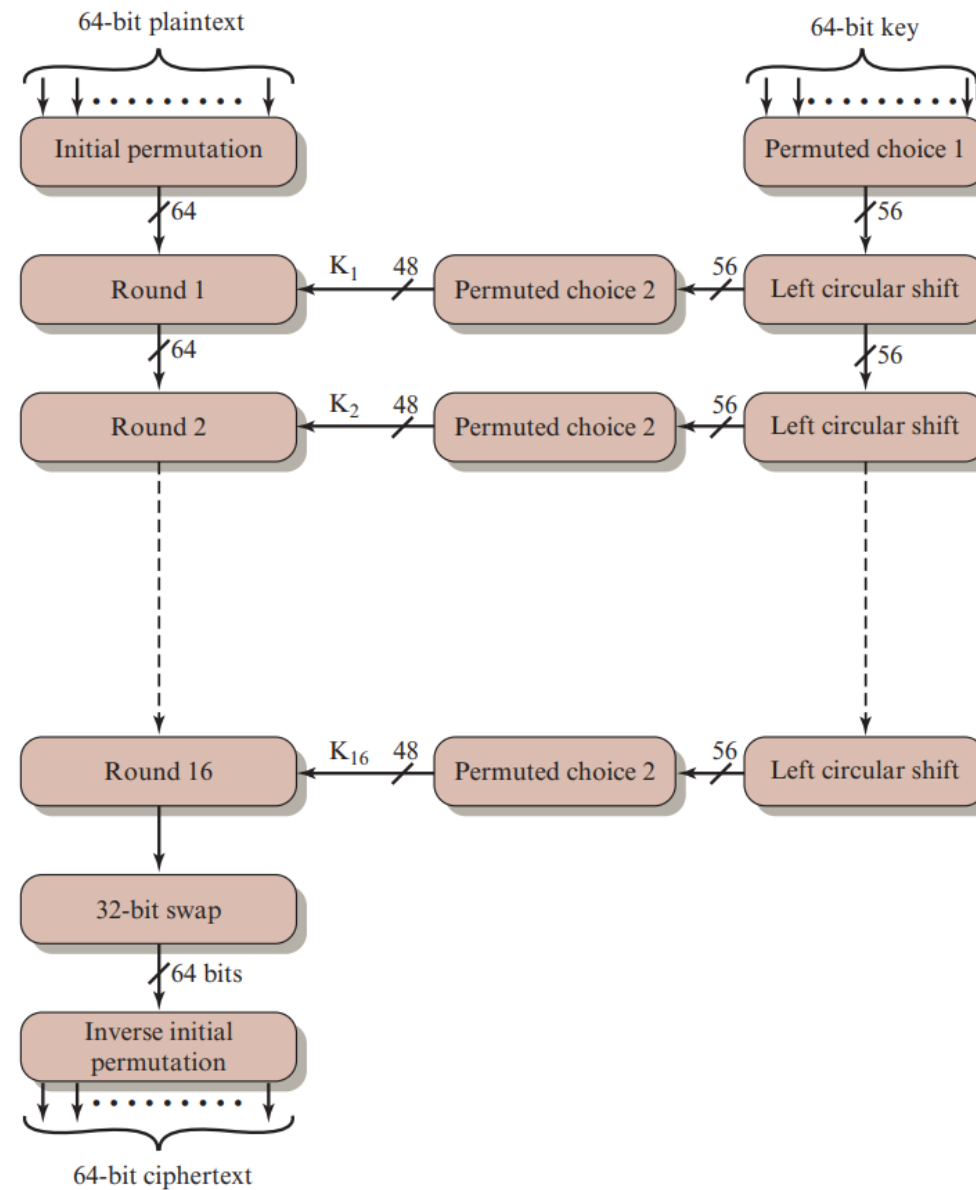


Figure 4.5 General Depiction of DES Encryption Algorithm

DES Decryption

Feistel Cipher Property

As with any Feistel cipher, decryption uses the same algorithm as encryption, except that the application of the subkeys is reversed.

This means that the decryption algorithm is structurally identical to the encryption algorithm.

The same hardware or software implementation can be used for both operations, with only the key schedule differing.

Decryption Steps

- 1 The ciphertext is first passed through the inverse of the final permutation used in encryption (which is the initial permutation IP).
- 2 The 16 rounds are applied in reverse order, using subkeys $K_{16}, K_{15}, \dots, K_1$ instead of $K_1, K_2, \dots, K_{16}$.
- 3 Finally, the inverse initial permutation (IP^{-1}) is applied, which corresponds to the initial permutation IP of encryption.

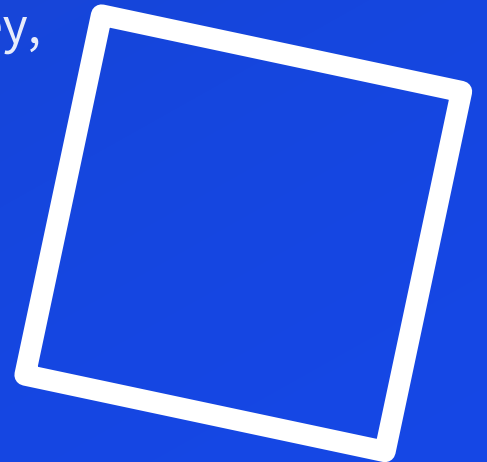
This elegant property of Feistel ciphers means that the same circuitry or code can be used for both encryption and decryption, simplifying implementation.



4.3

A DES Example

Working through a concrete DES example with specific plaintext, key, and resulting ciphertext values.



DES Example: Plaintext, Key, and Ciphertext

Example Introduction

We now work through an example and consider some of its implications.

Although you are not expected to duplicate the example by hand, you will find it informative to study the hex patterns that occur from one step to the next.

For this example, the plaintext is a hexadecimal palindrome.

Plaintext

02468ace
eca86420

(hexadecimal palindrome)

Key

0f1571c9
47d9e859

(56-bit effective key)

Ciphertext

da02ce3a
89ecac3b

(result after 16 rounds)

The following table shows the progression of the algorithm through all 16 rounds.

Table 4.2

DES Example Results

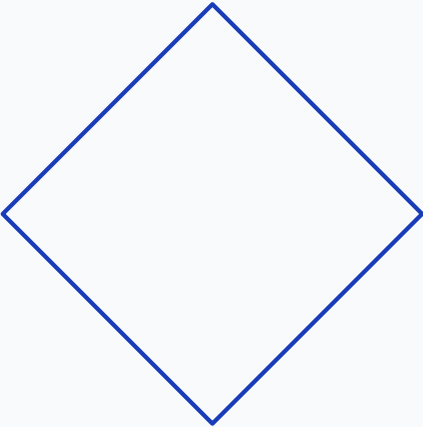


Table 4.2 DES Example

Round	K_i	L_i	R_i
IP		5a005a00	3cf03c0f
1	1e030f03080d2930	3cf03c0f	bad22845
2	0a31293432242318	bad22845	99e9b723
3	23072318201d0c1d	99e9b723	0bae3b9e
4	05261d3824311a20	0bae3b9e	42415649
5	3325340136002c25	42415649	18b3fa41
6	123a2d0d04262a1c	18b3fa41	9616fe23
7	021f120b1c130611	9616fe23	67117cf2
8	1c10372a2832002b	67117cf2	c11bfc09
9	04292a380c341f03	c11bfc09	887fbc6c
10	2703212607280403	887fbc6c	600f7e8b
11	2826390c31261504	600f7e8b	f596506e
12	12071c241a0a0f08	f596506e	738538b8
13	300935393c0d100b	738538b8	c6a62c4e
14	311e09231321182a	c6a62c4e	56b0bd75
15	283d3e0227072528	56b0bd75	75e8fd8f
16	2921080b13143025	75e8fd8f	25896490
IP ⁻¹		da02ce3a	89ecac3b

Note: DES subkeys are shown as eight 6-bit values in hex format

The Avalanche Effect

Definition

A desirable property of any encryption algorithm is that a small change in either the plaintext or the key should produce a significant change in the ciphertext.

In particular, a change in one bit of the plaintext or one bit of the key should produce a change in many bits of the ciphertext.

This is referred to as the avalanche effect.

Importance of Avalanche Effect

If the change were small, this might provide a way to reduce the size of the plaintext or key space to be searched.

An attacker could potentially exploit small changes to make cryptanalysis more efficient by tracking how small input changes affect the output.

The avalanche effect ensures that the cipher thoroughly mixes the input bits, making statistical analysis and differential cryptanalysis more difficult.

Table 4.3

Avalanche Effect in DES: Change in Plaintext

Table 4.3 Avalanche Effect in DES: Change in Plaintext

Round		δ
	02468aceeca86420 12468aceeca86420	1
1	3cf03c0fbad22845 3cf03c0fbad32845	1
2	bad2284599e9b723 bad3284539a9b7a3	5
3	99e9b7230bae3b9e 39a9b7a3171cb8b3	18
4	0bae3b9e42415649 171cb8b3ccaca55e	34
5	4241564918b3fa41 ccaca55ed16c3653	37
6	18b3fa419616fe23 d16c3653cf402c68	33
7	9616fe2367117cf2 cf402c682b2cefbc	32
8	67117cf2c11bfc09 2b2cefbc99f91153	33

Round		δ
9	c11bfc09887fbc6c 99f911532eed7d94	32
10	887fbc6c600f7e8b 2eed7d94d0f23094	34
11	600f7e8bf596506e d0f23094455da9c4	37
12	f596506e738538b8 455da9c47f6e3cf3	31
13	738538b8c6a62c4e 7f6e3cf34bc1a8d9	29
14	c6a62c4e56b0bd75 4bc1a8d91e07d409	33
15	56b0bd7575e8fd8f 1e07d4091ce2e6dc	31
16	75e8fd8f25896490 1ce2e6dc365e5f59	32
IP⁻¹	da02ce3a89ecac3b 057cde97d7683f2a	32

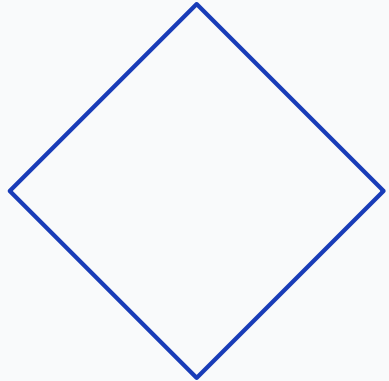


Table 4.4

Avalanche Effect in DES: Change in Key

Table 4.4 Avalanche Effect in DES: Change in Key

Round		δ
	02468aceeca86420 02468aceeca86420	0
1	3cf03c0fbad22845 3cf03c0f9ad628c5	3
2	bad2284599e9b723 9ad628c59939136b	11
3	99e9b7230bae3b9e 9939136b768067b7	25
4	0bae3b9e42415649 768067b75a8807c5	29
5	4241564918b3fa41 5a8807c5488dbe94	26
6	18b3fa419616fe23 488dbe94aba7fe53	26
7	9616fe2367117cf2 aba7fe53177d21e4	27
8	67117cf2c11bfc09 177d21e4548f1de4	32

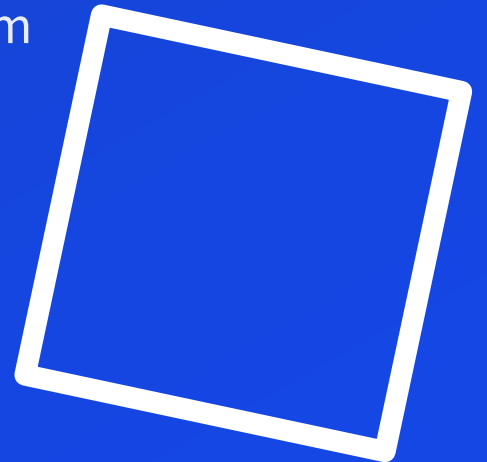
Round		δ
9	c11bfc09887fbc6c 548f1de471f64dfd	34
10	887fbc6c600f7e8b 71f64dfd4279876c	36
11	600f7e8bf596506e 4279876c399fdc0d	32
12	f596506e738538b8 399fdc0d6d208dbb	28
13	738538b8c6a62c4e 6d208dbbb9bdeea	33
14	c6a62c4e56b0bd75 b9bdeeaad2c3a56f	30
15	56b0bd7575e8fd8f d2c3a56f2765c1fb	27
16	75e8fd8f25896490 2765c1fb01263dc4	30
IP^{-1}	da02ce3a89ecac3b ee92b50606b62b0b	30



4.4

The Strength of DES

Analysis of DES security concerns focusing on key size and algorithm nature.



The Use of 56-Bit Keys

Key Space Size

With a key length of 56 bits, there are 2^{56} possible keys, which is approximately 7.2×10^{16} keys.

Thus, on the face of it, a brute-force attack appears impractical.

Assuming that, on average, half the key space has to be searched, a single machine performing one DES encryption per microsecond would take more than a thousand years to break the cipher.

Historical Perspective: Diffie-Hellman Analysis (1977)

As far back as 1977, Diffie and Hellman postulated that the technology existed to build a parallel machine with 1 million encryption devices.

Each device could perform one encryption per microsecond, which would bring the average search time down to about 10 hours.

The authors estimated that the cost would be about \$20 million in 1977 dollars.

Modern Computing Power

With current technology, it is not even necessary to use special, purpose-built hardware. The speed of commercial, off-the-shelf processors threatens the security of DES.

Modern Threats to DES Security

Contemporary Processing Speeds

A 2008 paper from Seagate Technology suggests that a rate of 1 billion (10^9) key combinations per second is reasonable for today's multicore computers.

Both Intel and AMD now offer hardware-based instructions to accelerate encryption.

Tests run on a contemporary multicore Intel machine resulted in an encryption rate of about half a billion encryptions per second.

Supercomputer Capabilities

Another recent analysis suggests that with contemporary supercomputer technology, a rate of 10^{13} encryptions per second is reasonable.

At this rate, DES can be broken in about an hour using a supercomputer.

Alternatives to DES

Fortunately, there are a number of alternatives to DES, the most important of which are AES and triple DES.

Table 4.5

Average Time Required for Exhaustive Key Search

Table 4.5 Average Time Required for Exhaustive Key Search

Key Size (bits)	Cipher	Number of Alternative Keys	Time Required at 10^9 Decryptions/s	Time Required at 10^{13} Decryptions/s
56	DES	$2^{56} \approx 7.2 \times 10^{16}$	$2^{55} \text{ ns} = 1.125 \text{ years}$	1 hour
128	AES	$2^{128} \approx 3.4 \times 10^{38}$	$2^{127} \text{ ns} = 5.3 \times 10^{21} \text{ years}$	$5.3 \times 10^{17} \text{ years}$
168	Triple DES	$2^{168} \approx 3.7 \times 10^{50}$	$2^{167} \text{ ns} = 5.8 \times 10^{33} \text{ years}$	$5.8 \times 10^{29} \text{ years}$
192	AES	$2^{192} \approx 6.3 \times 10^{57}$	$2^{191} \text{ ns} = 9.8 \times 10^{40} \text{ years}$	$9.8 \times 10^{36} \text{ years}$
256	AES	$2^{256} \approx 1.2 \times 10^{77}$	$2^{255} \text{ ns} = 1.8 \times 10^{60} \text{ years}$	$1.8 \times 10^{56} \text{ years}$
26 characters (permutation)	Monoalphabetic	$2! = 4 \times 10^{26}$	$2 \times 10^{26} \text{ ns} = 6.3 \times 10^9 \text{ years}$	$6.3 \times 10^6 \text{ years}$

The Nature of the DES Algorithm

Concerns About S-Box Design

Another concern is the possibility that cryptanalysis is possible by exploiting the characteristics of the DES algorithm.

The focus of concern has been on the eight substitution tables, or S-boxes, that are used in each iteration.

Because the design criteria for these boxes, and indeed for the entire algorithm, were not made public, there is a suspicion that the boxes were constructed in such a way that cryptanalysis is possible for an opponent who knows the weaknesses in the S-boxes.

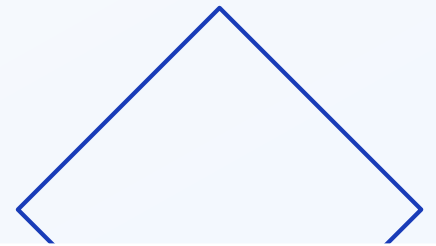
Current Status

This assertion is tantalizing, and over the years a number of regularities and unexpected behaviors of the S-boxes have been discovered.

Despite this, no one has so far succeeded in discovering the supposed fatal weaknesses in the S-boxes.

(At least, no one has publicly acknowledged such a discovery.)

Timing Attacks



What is a Timing Attack?

A timing attack is one in which information about the key or the plaintext is obtained by observing how long it takes a given implementation to perform decryptions on various ciphertexts.

A timing attack exploits the fact that an encryption or decryption algorithm often takes slightly different amounts of time on different inputs.

The execution time may vary based on the specific values being processed, potentially leaking information about the secret key.

Timing Attacks on DES

Research has reported on an approach that yields the Hamming weight (number of bits equal to one) of the secret key.

This is a long way from knowing the actual key, but it is an intriguing first step.

The authors conclude that DES appears to be fairly resistant to a successful timing attack but suggest some avenues to explore.

Although this is an interesting line of attack, it so far appears unlikely that this technique will ever be successful against DES or more powerful symmetric ciphers such as triple DES and AES.



4.5

Block Cipher Design Principles



Examination of three critical aspects: number of rounds, design of function F , and key scheduling.

Number of Rounds

Sources of Cryptographic Strength

The cryptographic strength of a Feistel cipher derives from three aspects of the design:

- The number of rounds
- The function F
- The key schedule algorithm

Choosing the Number of Rounds

The greater the number of rounds, the more difficult it is to perform cryptanalysis, even for a relatively weak F .

In general, the criterion should be that the number of rounds is chosen so that known cryptanalytic efforts require greater effort than a simple brute-force key search attack.

This criterion was certainly used in the design of DES.

DES Example

Schneier observes that for 16-round DES, a differential cryptanalysis attack is slightly less efficient than brute force: The differential cryptanalysis attack requires $2^{55} \cdot 1$ operations, whereas brute force requires 2^{55} . If DES had 15 or fewer rounds, differential cryptanalysis would require less effort than a brute-force key search.

Benefits of the Round Number Criterion

Advantages of This Criterion

This criterion is attractive because it makes it easy to judge the strength of an algorithm and to compare different algorithms.

In the absence of a cryptanalytic breakthrough, the strength of any algorithm that satisfies the criterion can be judged solely on key length.

This simplifies security analysis and provides a clear metric for comparing different encryption algorithms.

Practical Implications

When a cipher satisfies this criterion, security engineers can focus on key management and key length as the primary security factors.

The number of rounds provides a safety margin against future cryptanalytic advances.

Designers can confidently specify key lengths knowing that the round structure provides adequate protection against known attacks.

This design principle has stood the test of time and remains relevant for modern block cipher design.

Design of Function F

Role of Function F

The heart of a Feistel block cipher is the function F , which provides the element of confusion in a Feistel cipher.

Thus, it must be difficult to "unscramble" the substitution performed by F .

One obvious criterion is that F be nonlinear, as we discussed previously.

Nonlinearity

The more nonlinear F , the more difficult any type of cryptanalysis will be.

There are several measures of nonlinearity, which are beyond the scope of this book.

In rough terms, the more difficult it is to approximate F by a set of linear equations, the more nonlinear F is.

Avalanche Properties

We would like the algorithm to have good avalanche properties. Recall that, in general, this means that a change in one bit of the input should produce a change in many bits of the output.

Strict Avalanche Criterion (SAC)

Definition of SAC

A more stringent version of the avalanche property is the strict avalanche criterion (SAC).

SAC states that any output bit j of an S-box should change with probability $1/2$ when any single input bit i is inverted for all i, j .

This means that changing any single input bit should cause each output bit to change with exactly 50% probability.

Application of SAC

Although SAC is expressed in terms of S-boxes, a similar criterion could be applied to F as a whole.

This is important when considering designs that do not include S-boxes.

The SAC ensures that the function F thoroughly mixes the input bits, making it difficult to trace input-output relationships.

SAC provides a quantitative measure of how well the function F spreads the influence of individual input bits across the output.

Bit Independence Criterion (BIC)

Definition of BIC

Another criterion proposed is the bit independence criterion (BIC).

BIC states that output bits j and k should change independently when any single input bit i is inverted for all i, j , and k .

This means that the changes in different output bits should not be correlated with each other when a single input bit changes.

Significance of BIC

The SAC and BIC criteria appear to strengthen the effectiveness of the confusion function.

Together, these criteria ensure that:

- Each output bit depends on many input bits (diffusion)
- The relationship between input and output is complex (confusion)
- Output bits change independently of each other (BIC)
- Each output bit changes with 50% probability for any input change (SAC)

These criteria have become fundamental principles in the design of modern block ciphers, ensuring strong cryptographic properties.



Acknowledgement

The material presented in these slides incorporates content from books, academic lecture notes, online resources, and generated text. All rights remain with the original authors and publishers. This content is used solely for educational purposes to illustrate and clarify concepts. No copyright infringement is intended.

Primary Content from: Cryptography and Network Security - William Stallings - 8e – Chapter 4

Thank You

Questions & Discussion